สรุปภาพรวมงาน: ความแข็งแกร่งของแต่ละประเทศในกีฬาปิงปอง

02739452 แพลตฟอร์มข้อมูลขนาดใหญ่และการวิเคราะห์ · สอบปลายภาค ภาคต้น 2569
ลำดับที่ 13 · รหัสนิสิต 6621600941 · ชื่อ-นามสกุล นฤบดินทร์ แอตาล หมู่ 700
GitHub: https://github.com/Naruebodin-Aetal/02739452-13-6621600941

1. ภาพรวมงานแต่ละส่วน

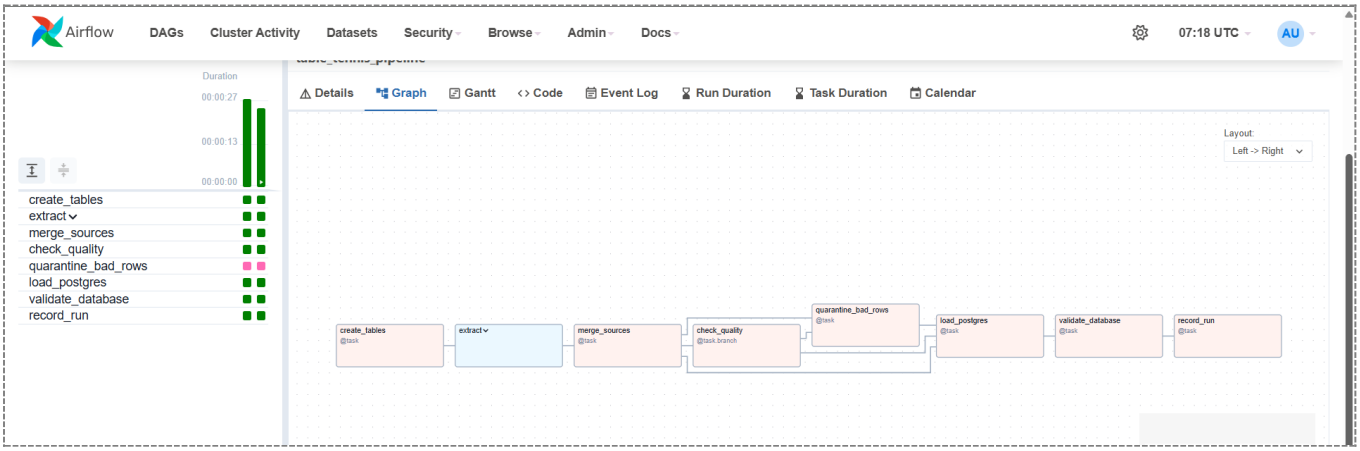
หัวข้อ: รวมเหรียญรางวัลปิงปองชิงแชมป์โลก เหรียญโอลิมปิก และอันดับโลก WTT เป็นข้อมูลรายประเทศ ด้วย Apache Airflow → PostgreSQL → เว็บ dashboard (Streamlit) ที่อ่านจาก PostgreSQL โดยตรง รันทั้งหมดด้วย docker compose

แหล่งข้อมูล	วิธีดึง	ข้อมูลที่ได้	แถวที่ได้
Wikipedia: World Championships medalists	Web scraping	ตารางเหรียญรายประเทศ 5 ประเภท (เดี่ยว/คู่/คู่ผสม)	113
Olympedia: Table Tennis	Web scraping	ตารางเหรียญโอลิมปิกตลอดกาลตามรหัส NOC	12
WTT Rankings API	REST API (ไม่ใช้ key)	อันดับโลก 1,000 อันดับแรก ชาย/หญิงเดี่ยว รวมเป็นรายประเทศ	242

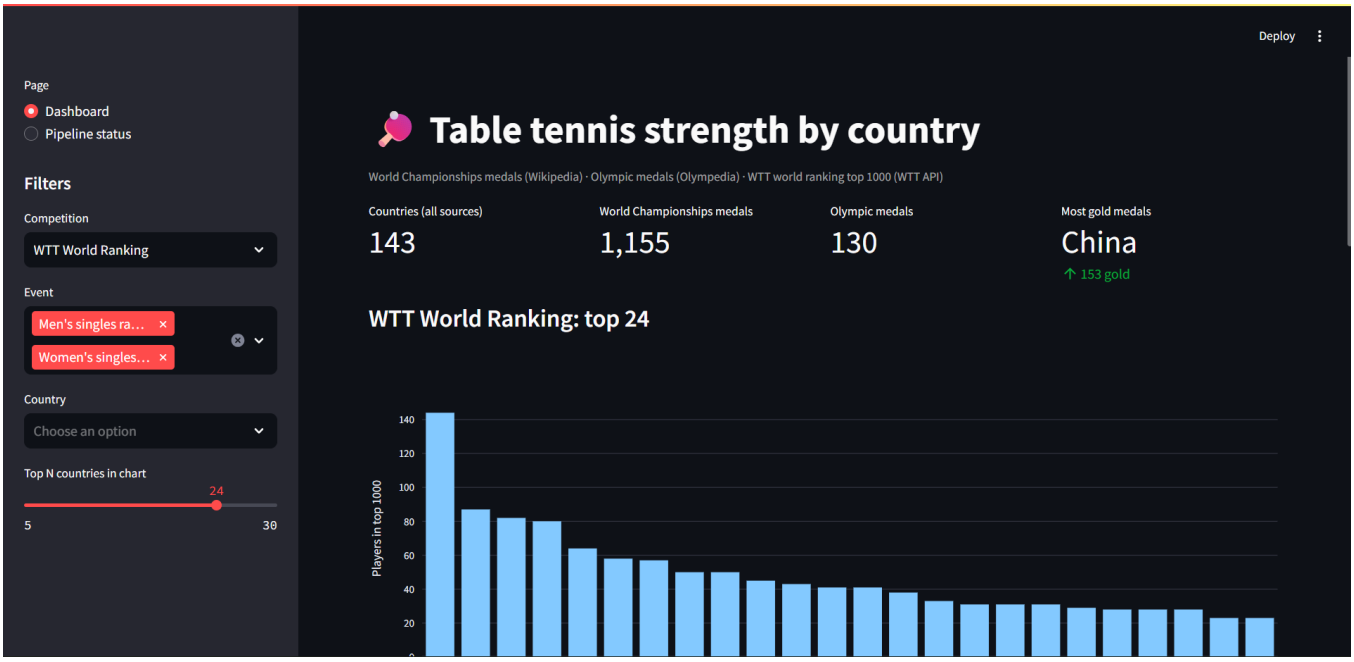
ข้อกำหนด: ไม่ใช้ Selenium (ใช้ requests + BeautifulSoup) · ตรวจ robots.txt ในโค้ดก่อน scrape ทุกครั้ง · PDPA: ตัดชื่อ/รหัสนักกีฬาจาก WTT ตั้งแต่ขั้นดึง เก็บเฉพาะข้อมูลรายประเทศ · ข้อมูลรวม 367 แถว 143 ประเทศ

ระดับ	สิ่งที่ทำได้	สถานะ
1 ขั้นต่ำ	DAG extract → transform → load → PostgreSQL · เว็บแสดงตารางและกราฟ	ผ่าน
2 มาตรฐาน	3 แหล่งรันขนานใน TaskGroup extract · schema เดียว (country_stats) · UPSERT ด้วย ON CONFLICT รันซ้ำ 4 รอบยัง 367 แถว · KPI 4 ตัว + ตัวกรอง 4 แบบ	ผ่าน
3 ท้าทาย	merge_sources ใช้ trigger_rule=all_done จึงรวมแหล่งที่เหลือได้เมื่อแหล่งใดล่ม · ตัดซ้ำข้ามแหล่งด้วยรหัสประเทศ (ตาราง countries) · check_quality แตกเส้นทางไป quarantine_bad_rows เมื่อไม่ผ่าน	ผ่าน (ทดสอบจริง)
โบนัส	Dynamic Task Mapping: fetch_wtt_ranking.expand([MS, WS]) · หน้า Pipeline status บนเว็บ	ผ่านทั้งสองอย่าง

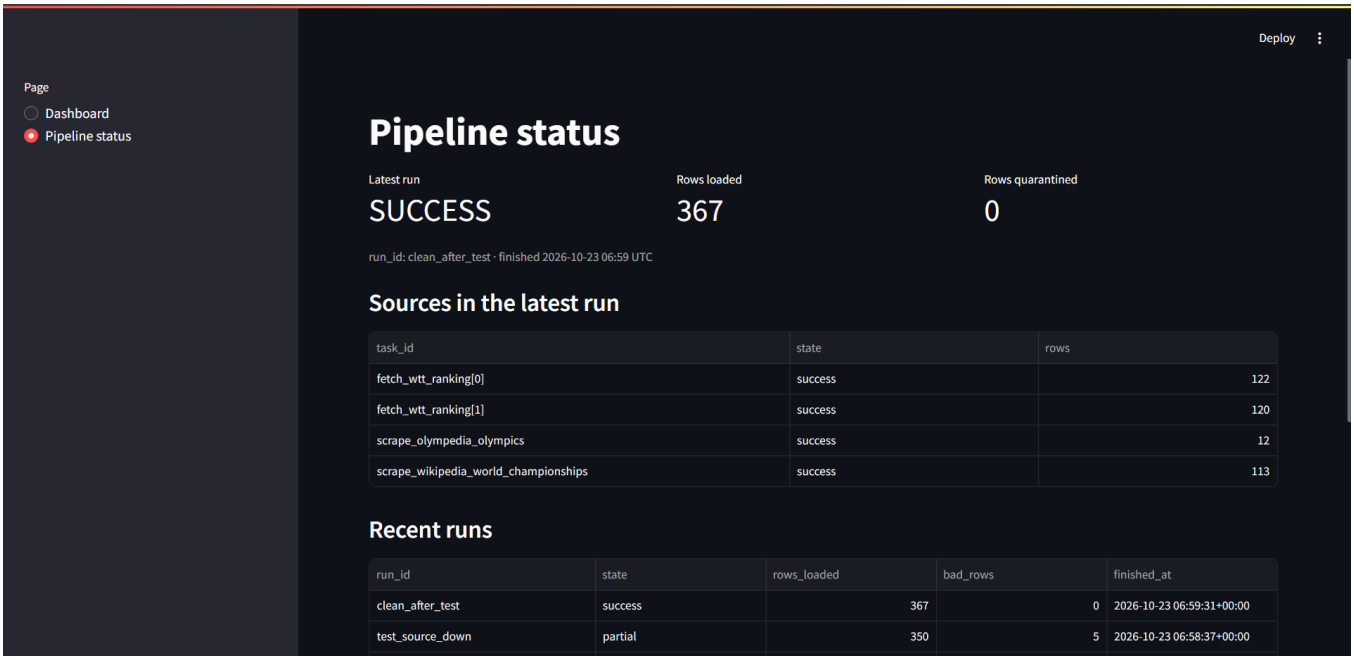
ลำดับงานใน DAG: create_tables → extract (TaskGroup: scrape_wikipedia, scrape_olympedia, fetch_wtt_ranking[MS, WS]) → merge_sources → check_quality (branch) → [quarantine_bad_rows] → load_postgres (UPSERT) → validate_database (≥100 แถว, ไม่มีคีย์ซ้ำ) → record_run (บันทึกสถานะสำหรับหน้า Pipeline status)



ภาพที่ 1 Graph view ของ DAG table_tennis_pipeline



ภาพที่ 2 หน้า Dashboard (http://localhost:8501)



ภาพที่ 3 หน้า Pipeline status แสดงสถานะรายแหล่งของรอบล่าสุด

2. ปัญหาที่พบและวิธีแก้

ปัญหาที่ 1: Wikipedia มี "ครึ่งเหรียญ" ทำให้ข้อมูลไม่ผ่านการตรวจคุณภาพ

- **อาการ:** ทดสอบตัวดึงข้อมูลครั้งแรก 38 แถวจากตารางประเภทคู่ไม่ผ่านกฎ invalid_medal_count ทั้งที่ประเภทเดี่ยวผ่านหมด
- **หาสาเหตุ:** พิมพ์ค่าดิบในเซลล์ของตารางชายคู่ออกมาดู พบค่าเช่น 11.5 และ 34.5
  เพราะคู่ที่ผู้เล่นมาจากสองประเทศแบ่งเหรียญกันคนละครึ่ง ตัวแปลงเดิมรับเฉพาะจำนวนเต็มจึงได้ค่าว่าง
- **วิธีแก้:** แปลงเป็นทศนิยมและเปลี่ยนคอลัมน์เหรียญเป็น NUMERIC(6,1) หลังแก้ ทั้ง 367 แถวผ่าน และกฎ ทอง+เงิน+ทองแดง = รวม ยังตรวจได้ถูกต้อง

ปัญหาที่ 2: ชื่อประเทศไม่ตรงกันข้ามแหล่ง ทำให้ประเทศเดียวกันถูกนับซ้ำ

- **อาการ:** จีนปรากฏเป็น "China" (Wikipedia), "People's Republic of China" (Olympedia) และรหัส CHN (WTT)
  ถ้ารวมด้วยชื่อจะได้ 3 ประเทศแยกกัน
- **หาสาเหตุ:** เทียบคอลัมน์ของแต่ละแหล่ง พบว่า Olympedia และ WTT ให้รหัส 3 ตัวอยู่แล้ว มีเพียง Wikipedia ที่ให้แค่ชื่อ และมีทีมในอดีต เช่น Soviet Union, West Germany, Czechoslovakia
- **วิธีแก้:** ทำตารางแปลงชื่อ Wikipedia เป็นรหัส (ทีมในอดีตใช้รหัสของตัวเอง เช่น URS, FRG, TCH) แล้วรวมเป็นหนึ่งแถวต่อรหัสในตาราง countries แถวที่แปลงไม่ได้จะถูกส่งไป quarantine_bad_rows ทดสอบโดยลบชื่อ England ออกจากตารางแปลงชั่วคราว พบว่า 5 แถวของ England ไปอยู่ใน quality_issues และแถวอื่นโหลดตามปกติ

ปัญหาที่ 3: ต้องพิสูจน์ว่า DAG ยังทำงานต่อเมื่อแหล่งข้อมูลล่ม

- **อาการ:** ทุกเว็บใช้งานได้ปกติ จึงยังไม่เคยเห็นเส้นทางที่แหล่งใดแหล่งหนึ่งล้มเหลว และถ้าใช้ trigger rule ค่าเริ่มต้น (all_success) task ถัดไปจะถูกข้ามทั้งหมด
- **หาสาเหตุ/ทดสอบ:** บล็อกโดเมน www.olympedia.org ใน container ของ Airflow ชั่วคราว แล้ว trigger รอบ test_source_down
- **วิธีแก้/ผล:** ตั้ง merge_sources เป็น trigger_rule=all_done และอ่านเฉพาะผลของแหล่งที่สำเร็จ ผลคือ scrape_olympedia_olympics ขึ้น failed แต่ DAG โหลด Wikipedia และ WTT ต่อจนจบ ข้อมูล Olympedia ของรอบก่อนยังอยู่เพราะใช้ UPSERT และหน้า Pipeline status แสดงสถานะ PARTIAL

3. การวางแผนและจัดลำดับงาน

ช่วงเวลา	งาน	เหตุผล
13:00–13:30	เลือกหัวข้อ ตรวจ robots.txt และทดสอบว่าแต่ละแหล่งดึงได้จริงโดยไม่ต้องใช้ JavaScript	ตัดแหล่งที่ใช้ไม่ได้ทิ้งก่อนเขียนโค้ด
13:30–13:45	เขียนตัวดึงข้อมูลและชั้น transform แล้วทดสอบกับเว็บจริงนอก Airflow	แก้ปัญหาข้อมูล (ครึ่งเหรียญ) ได้เร็วกว่าดีบักใน DAG
13:45–14:00	เขียน DAG, schema, dashboard และ docker compose แล้วรันทั้งระบบ	ทำระดับ 1–2 ให้ครบก่อน แล้วค่อยเติมระดับ 3
14:00–14:10	ทดสอบรันซ้ำ (UPSERT), จำลองแหล่งล่ม และเส้นทาง quarantine แล้ว push ขึ้น GitHub	พิสูจน์เงื่อนไขระดับ 3 ด้วยการรันจริง
14:10–15:30	แคปภาพและเขียนรายงาน PDF เผื่อเวลาแก้ไข	รายงานมีคะแนน 35 คะแนน

งานที่ตัดออกและเหตุผล:

- **อันดับโลกประเภทคู่และรุ่นเยาวชน:** WTT API คืนค่า 0 แถวสำหรับ MD และ U19 จึงใช้เฉพาะชายเดี่ยวและหญิงเดี่ยว
- **ข้อมูลรายนักกีฬา:** ชื่อและรหัสนักกีฬาเป็นข้อมูลส่วนบุคคลตาม PDPA จึงรวมเป็นรายประเทศแทน
- **แหล่งอื่น เช่น ITTF Results และหน้าเว็บ WTT:** ITTF Results ตอบกลับ 429 (ถูกจำกัดการเรียก) และเว็บ WTT เป็น JavaScript ทั้งหน้า ซึ่งต้องใช้ Selenium ที่โจทย์ห้าม
- **การแจ้งเตือนทางอีเมลเมื่อ DAG ล้มเหลว:** ต้องตั้ง SMTP และไม่อยู่ในเกณฑ์คะแนน ใช้หน้า Pipeline status แทน